

HHM

QwanxingxuA

2026 年 9 月 1 日

1 隐马尔可夫模型

从学习朴素贝叶斯、EM 算法可以发现这一系列算法都在解决原问题不易优化的问题。隐马尔可夫也是如此。

EM 算法有一个没被说明但默认的前提：隐变量 $\mathbf{z}$ 独立同分布。这个假设是很强的，在以前的算法中我们努力找到这种分布，如 EM 算法的 $P(\mathbf{x}, \mathbf{z})$ 或者变分的 $\tilde{P}(\mathbf{z})$ 。

那么能不能不假设独立同分布呢。这个时候就不能像 EM 算法去寻找随机变量 $\mathbf{z}$ 的分布了，而是 $z_1, z_2, \dots, z_T$ 各自的分布了。但是大前提 $\mathbf{x}$ 独立同分布是不能不假设的。

从直观的角度看，观测数据的时候往往不是一次观测的，而是随着时间逐渐观测的。那对应的隐变量也是随着时间产生的，在时间维度上或许存在在联系。这种联系是很难界定的，机器学习解决的问题之一就是寻找原始解空间的子解空间并在该空间找到最优解。从具体的方法而言就是大胆假设。假设这种联系与时间间隔有关，隐马尔可夫则是更加强的假设：只与前一时间有关。

用数学写出以上的假设。假设观测序列 $\mathbf{x} = x_1 x_2 \dots x_T$ ，状态序列 $\mathbf{z} = z_1 z_2 \dots z_T$ 。假设隐变量的所有可能集合：

$$\mathcal{S} = \{s_1, s_2, \dots, s_I\}$$

观测数据的所有可能集合：

$$\mathcal{O} = \{o_1, o_2, \dots, o_K\}$$

强调一点， $s \in \mathcal{S}$ 是向量，因为对于某一时刻的隐变量不一定是一个，就算是一个也可以通过独热编码转化，都统一成向量。

则上述假设即马尔可夫性：

$$P(\mathbf{z}_t | \mathbf{z}) = P(\mathbf{z}_t | \mathbf{z}_{<t})$$

现在考虑原始概率问题, 将以前的写法写作：

$$P_\lambda(\mathbf{x}) = \sum_{\mathbf{z}} P_\lambda(\mathbf{z}) P_\lambda(\mathbf{x} | \mathbf{z})$$

其中, λ 表示参数。将表达式写成与时间有关的表达式：

$$P_\lambda(\mathbf{x}_t) = \sum_{\mathbf{z}} P_\lambda(\mathbf{z}_t) P_\lambda(\mathbf{x}_t | \mathbf{z}_t)$$

第一项好解决, 基于马尔可夫性：

$$\begin{aligned} P_\lambda(\mathbf{z}_t) &= P_\lambda(z_1, z_2, \dots, z_t) \\ &= P_\lambda(z_1) P_\lambda(z_2 | z_1) \cdots P_\lambda(z_t | \mathbf{z}_{<t}) \\ &= P_\lambda(z_1) P_\lambda(z_2 | z_1) \cdots P_\lambda(z_t | z_{t-1}) \end{aligned}$$

第二项就有点困难了, 观测数据与状态的联系怎么恒定。为了简单, 提出一个假设即观测独立性：观测数据只与同时刻的状态有关：

$$P_\lambda(x_t | \mathbf{z}_t) = P_\lambda(x_t | z_t)$$

所以第二项：

$$P_\lambda(\mathbf{x}_t | \mathbf{z}_t) = P_\lambda(x_1 | z_1) P_\lambda(x_2 | z_2) \cdots P_\lambda(x_t | z_t)$$

可以得到分布概率矩阵：

$$A_{ij} = P_\lambda(z_t = s_j | z_t = s_i), \quad i = 1, 2, \dots, I \quad j = 1, 2, \dots, I$$

$$B_{is} = P_\lambda(x_t = o_s | z_t = s_i) \quad i = 1, 2, \dots, I \quad s = 1, 2, \dots, K$$

分布概率矩阵其实就是我们的参数, 即关于隐变量的模型参数

$$\lambda = (A, B)$$

原似然函数变为：

$$P_\lambda(\mathbf{x}) = \sum_{\mathbf{z}} [P_\lambda(z_1) P_\lambda(x_1 | z_1)] [P_\lambda(z_2 | z_1) P_\lambda(x_2 | z_2)] \cdots [P_\lambda(z_t | z_{t-1}) P_\lambda(x_t | z_t)]$$

优化上述问题即可。下看怎么优化。

直接优化上述问题是个巨大工程。因为矩阵 A, B 有时间 T 个，一次性学习好像无法入手；每个矩阵都要存储消耗资源极大。

解决第一个难点就是添加一个参数 π ，它是矩阵 A, B 的初始值，即第一个隐变量分布。没有初始值的话这个矩阵难以入手，至于初始值是怎么样的，我们先假设有这样一个初始值，将其视为参数。

解决第二个难点就是使用序列。可以发现矩阵 A_t, B_t 只与前一时间有关，再者我们模型的预测也是拿到矩阵 A_t, B_t 开始预测的。换言之，根本不需要存储所有矩阵，只需要存储当前时间的矩阵。则

$$\lambda = (A, B, \pi)$$

不难发现，上述就是动态规划的思想。矩阵 A, B 的优化过程具 T 有最优子结构性质和无后效性。

上述思想也可以反过来。于是有了前向、后向方法以及它们结合的方法。具体算法略。

2 Baum-Welch 与预测算法

Baum-Welch 方法是与 EM 算法的结合。以上的推导与 EM 算法的推导过程并不矛盾。代入结论：

$$Q(\lambda, \bar{\lambda}) = \sum_z P_{\bar{\lambda}}(z | \mathbf{x}) \log P_{\lambda}(\mathbf{x}, z)$$

$$P_{\lambda}(\mathbf{x}, z) = P(z_1)P(x_1 | z_1)P(z_2 | z_1)P(x_2 | z_2) \dots P(z_t | z_{t-1})P(x_t | z_t)$$

EM 算法不在赘述，对每一个参数构建拉格朗日函数求导即可。

预测算法不计算 z 的具体分布，而是求取最大的路径。马尔可夫可以转换成概率图的形式，并且马尔可夫只与前一时间有关，在图上面，假设以时间为轴展开，具有类似与神经网络的形状，这是非常好的。

基于图论知识，有三种方法可以解决。暴力枚举：这个就不用多说了吧；近似算法：每一步都取最大的边，依次走下去，这种启发式算法简单但是并不一定能找到最大路径；维特比算法：动态规划算法，原理参考图论的动态规划问题。

以上算法笔者只是浅浅而谈，具体看实操。

3 EM 算法与 Baum-Welch 预测算法的不同

有没有想过李航老师书中为什么不写

$$Q(\lambda, \bar{\lambda}) = \mathbb{E}_{P_{\bar{\lambda}}(\mathbf{z}|\mathbf{x})} \log P_{\lambda}(\mathbf{x}, \mathbf{z})$$

因为这是 EM 算法的通用表达式。这个表达式在 Baum-Welch 中需要变化一下:

$$Q(\lambda, \bar{\lambda}) = \mathbb{E}_{P_{\bar{\lambda}}(\mathbf{z}_t|\mathbf{x})} \log P_{\lambda}(\mathbf{x}, \mathbf{z}_t) \neq \mathbb{E}_{P_{\bar{\lambda}}(\mathbf{z}|\mathbf{x})} \log P_{\lambda}(\mathbf{x}, \mathbf{z})$$

不同就在于 $\mathbf{z}$ 不再独立同分布, 这也意味着不能再直接求解完全数据的似然概率

$$P_{\lambda}(\mathbf{x}, \mathbf{z})$$

就算使用也是计算

$$P_{\lambda}(\mathbf{x}, \mathbf{z}_t)$$

4 反思深度学习

笔者是在了解过深度学习之后仔细推导这篇笔记的。深度学习特别是自然语言部分与介绍的马尔可夫息息相关。

自然语言处理中词嵌入就是使用了类似于隐马尔可夫的思想。其变化是观测数据受影响的时间长度, 还有表达式的线性化, 实际上这是因为将概率模型通过指数分布转化成了线性的形式, 本质是一样的。

以上思想用在了文本生成。基于时间序列的思想, 可以由一句话预测出下一个词, 这其中是通过隐变量 h 传递信息的。这中模型直观上是很自然的。

机器翻译是深度学习重要的应用。编码器解码器是机器翻译的一种方法, 以中译英为例: “我爱你” 翻译成 “I love you”。通过编码器生成的隐变量作为解码器的输入, 笔者常常将其理解为语义传给解码器。但是笔者现在总觉得怪怪的, 这种方式和直接学习 “我爱你 I love you” 有什么区别, 笔者常常觉得这不是机器翻译而是学习了一门中英文混合的新语言文本生成器。“I love you” 通过隐变量传递生成很自然, 但是一一翻译的话, “love” 应该与 “我” “我爱” “我爱你” 都有关系, 是整体翻译而来。但是解码器编码器的机器翻译在很多场景下效果有很好, 笔者只能换一种角度理解: 受马尔

可夫启发，编码器学习到了解码器的初始参数，因为初始值是这种时序模型很关键的一点。从这个角度来看，似乎也合理。

门语言模型对传递的信息变量做了取舍。这对机器翻译来说肯定是非常大的优化，因为翻译的过程，不可能考虑所有信息。但是门语言模型依旧是通过隐变量连接，笔者的第一种看法的问题依旧没有解决。

其实笔者的疑问就是上述模型似乎就是文本生成器，不是翻译器。问题出在翻译受原中文句子的影响很大，机器翻译应该要考虑才能不叫作文本生成器。注意力机制帮笔者解决了这一问题，把原中文句子学习的各隐变量纳入门语言模型。加入了注意力机制的机器翻译模型性能是非常强大的。

但是笔者始终觉得这种类似于文本生成器的模型并不是机器翻译模型，即使它们也能完成机器翻译任务。换言之，完全使用注意力机制搭建机器翻译模型才算是真正的机器翻译模型，当然 Transformer 通过搭建自注意力机制实现了这一点，并且取得了非常好的性能。注意力是所有你需要的！

以上只是笔者学习深度学习的一些困惑，在写这篇笔记的时候有些许想法，将它们写在了最后。当然，以上的 seq2seq、LSTM、GRU 等模型都是好模型，它们也远不是笔者上面所说的这么单一、简单，实际上还是有很大机制巧思的。